# Dynamic Programming

→ complex problems are divided to subproblems & the answers are stored in table or array, thus reduces computational time.

→ Solve the subproblem & store it for future usage.

→ Solutions are stored in table or array {Memoization} or in a bottom-up manner {tabulation} to avoid redundant computation. {infinite looping}

→ Memoization ⇒ Top-down approach using recursion
→ Tabulation ⇒ Bottom-up approach where solutions to subproblems are stored iteratively in a table.

## * Assembly Line Scheduling Problem-

- Automobile factory has two assembly lines

→ Each line has n stations : $S_{1,1}, \ldots, S_{1,n}$ & $S_{2,1}, \ldots, S_{2,n}$.

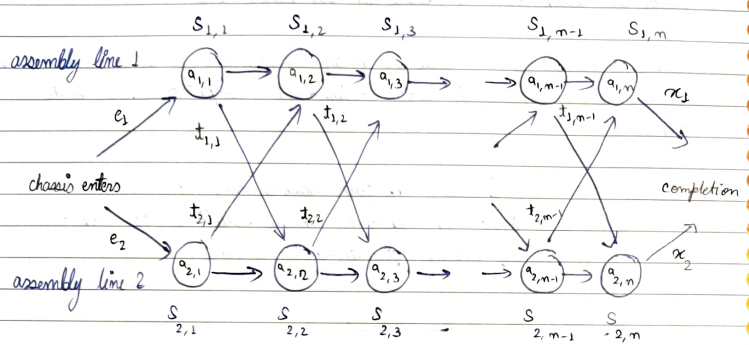
→ Corresponding stations $S_{1,j}$ & $S_{2,j}$ perform the same function but can take different amounts of time $a_{1,j}$ & $a_{2,j}$.

→ Entry times are $e_1$ & $e_2$ & exit times are $x_1$ & $x_2$.

→ After going through a station, car can either :

- stay on same line at no cost, or
- transfer to other line : cost after $S_{i,j}$ is $t_{i,j}$, $j = 1$ to $n-1$

→ Problem: what stations should be chosen from line 1 and line 2 in order to minimize the total time through the factory for one car?




→ how to get to $S_{1,j}$

- through $S_{1,j-1}$ then directly to $S_{1,j}$
- through $S_{2,j-1}$, then transfer over to $S_{1,j}$

Solution: Definitions:

$f^*$ → fastest time to get through the entire factory.

$f_i[j]$ → fastest time to get from starting pt. through station $S_{i,j}$



$l^*$ → line no. used to exit the factory from $n^{th}$ station.

$l_i[j]$ → line no. whose $S_{i,j-1}$ is used to reach $S_{i,j}$.

Objective function: $f^* = \min(f_1[n] + x_1, f_2[n] + x_2)$

Recursion:

~~Base Case~~:

$$f_1[j] = \begin{cases} e_1 + a_{1,1} & \Rightarrow \text{if } j = 1 \quad \text{(Base-case)} \\ \min(f_1[j-1] + a_{1,j},\ f_2[j-1] + t_{2,j-1} + a_{1,j}) & \end{cases}$$

$\Rightarrow$ if $j \geq 2$ (general case)

$$f_2[j] = \begin{cases} e_2 + a_{2,1} & \Rightarrow \text{if } j = 1 \quad \text{(Base-case)} \\ \min(f_2[j-1] + a_{2,j},\ f_1[j-1] + t_{1,j-1} + a_{2,j}) & \Rightarrow \text{if } j \geq 2 \quad \text{(general-case)} \end{cases}$$

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $f_1[j]$ | $f_1(1)$ | $f_1(2)$ | - | - | - | - | - |
| $f_2[j]$ | $f_2(1)$ | $f_2(2)$ | - | - | - | - | - |

j →

| | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| $l_1[j]$ | $l_1(1)$ | - | - | - | - | - | - |
| $l_2[j]$ | $l_2(1)$ | - | - | - | - | - | - |

## * Pseudo-Code { Time-complexity = O(n) }

```
FASTEST-WAY( a, t, e, x, n)
    f1[1] ← e1 + a1,1
    f2[1] ← e2 + a2,1

    for j ← 2 to n
        do if (f1[j-1] + a1,j ≤ f2[j-1] + t2,j-1 + a1,j)
            then (f1[j] ← f1[j-1] + a1,j
                  l1[j] ← 1 )

            else ( f1[j] ← f2[j-1] + t2,j-1 + a1,j
                   l1[j] ← 2 )

        if (f2[j-1] + a2,j ≤ f1[j-1] + t1,j-1 + a2,j )
            then f2[j] ← f2[j-1] + a2,j
                 l2[j] ← 2

            else f2[j] ← f1[j-1] + t1,j-1 + a2,j
                 l2[j] ← 1

if ( f1[n] + x1 ≤ f2[n] + x2 )
    then f* = f1[n] + x1
         l* = 1

    else f* = f2[n] + x2
         l* = 2
```

## * Printing stations :

```
Print-Stations ( l, n)
    i ← l*
    print "line" i ", station" n
    for j ← n down to 2
        do i ← li[j]
           print "line" i ", station" j-1
```

## * Matrix Chain Multiplication

for a given k value, pattern-

$$P(n) = \begin{cases} 1 & \text{if } n = 1 \\ \sum_{k=1}^{n-1} P(k) \cdot P(n-k) & \text{if } n \geq 2 \end{cases}$$ } Recurrence relation

P(n) = no. of alternative parenthesizations of a sequence of n matrices

if ($n \geq 2$) => a fully parenthesized matrix product is the product of two fully parenthesized matrix subproducts.

& the split occur between the $k^{th}$ & $(k+1)^{st}$ matrices for any $k = 1, 2, \ldots, n-1$.

The time complexity for this recursion = $\Omega(2^n)$

DOMS

DOMS

* Multiplication of 2 Matrices.

MATRIX- MULTIPLY (A, B)

if A.columns ≠ B.rows
    error "incompatible dimensions"

else let C be a new (A.rows x B.columns) matrix

    for i = 1 to A.rows
        for j = 1 to B.columns
            $c_{ij} = 0$

            for k = 1 to A.columns
                $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$

    return C

* Dynamic Programming Approach.

→ $P_m = A_1 \ldots\ldots A_m$ (storing the dimensions)

→ if parenthesis is at $k^{th}$ position, no. of multiplications

$$m[i,j] = m[i,k] + m[k+1, j] + pqr$$

size = p x q  size = q x r



$$m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} \{ m[i,k] + m[k+1,j] + p_{i-1} p_k p_j \} & \text{if } i < j \end{cases}$$

* PSEUDO - CODE

MATRIX-CHAIN-ORDER (p)

    n = p.length - 1
    let m = [1...n, 1...n] & s[1...n-1, 2...n] be new tables

    for i = 1 to n
        m[i,i] = 0

    for l = 2 to n        { l is the chain length }
        for i = 1 to n-l+1
            j = i+l-1
            m[i,j] = ∞
            for k = i to j-1
                q = m[i,k] + m[k+1,j] + $p_{i-1} p_k p_j$

                if (q < m[i,j])
                    m[i,j] = q
                    s[i,j] = k

    return m & s

* time-complexity ⇒ $O(n^3)$
* space-complexity ⇒ $O(n^2)$ for m & s

Ex-

| m | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 120 | 88 | (158) Ans |
| 2 | 0 | | 0 | 48 | 104 |
| 3 | 0 | | | 0 | 84 |
| 4 | 0 | | | | 0 |

| s | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 | 3 |
| 2 | 0 | | 0 | 2 | 3 |
| 3 | 0 | | | 0 | 3 |
| 4 | 0 | | | | 0 |

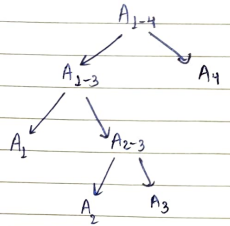
p =

| 5 | 4 | 6 | 2 | 7 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 |

, n = 5 - 1 = 4

$m[1,4] = \min \begin{cases} m[1,1] + m[2,4] + P_0P_1P_4 = 0 + 104 + 140 = 244 \\ m[1,2] + m[3,4] + P_0P_2P_4 = 120 + 84 + 210 \\ m[1,3] + m[4,4] + P_0P_3P_4 = 88 + 0 + 70 = (158) \end{cases}$

$A_1 A_2 A_3 A_4 \Rightarrow$ $(A_1)(A_2A_3A_4)$ (optimal one)
$(A_1A_2)(A_3A_4)$
$(A_1A_2A_3)(A_4)$




order of multiplication



* Longest Common Subsequence

→ string 1 : abcdefghi
→ string 2 : cdgi

* using recursion-

```
int LCS (i, j)
{
    if ( A[i] == '\0' || B[j] == '\0' )
        return 0 ;

    else if ( A[i] == B[j] )
        return 1 + LCS ( i+1, j+1 );

    else
        return max( LCS(i+1, j) , LCS(i, j+1)) ;
}
```

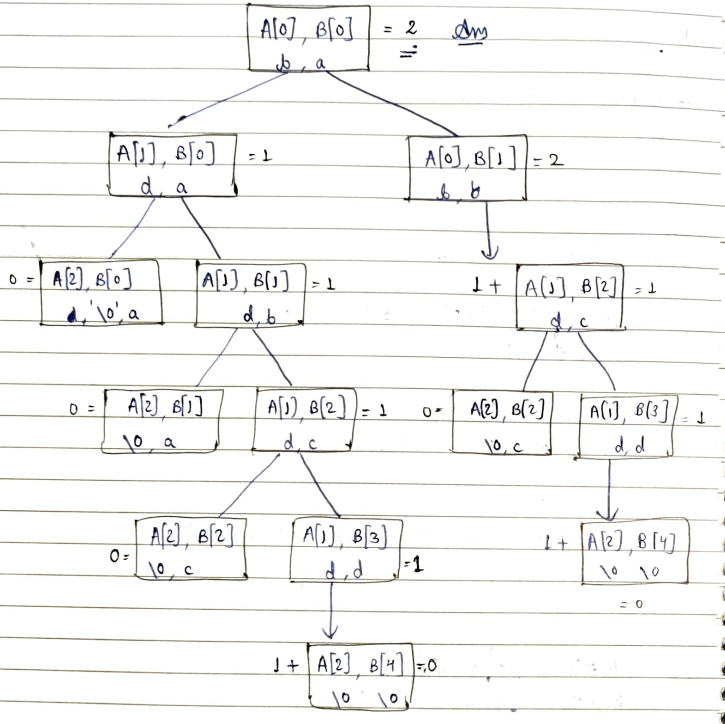
Ex- A ⇒

| b | d | \0 |
|---|---|---|

B ⇒

| a | b | c | d | \0 |
|---|---|---|---|---|

A[0], B[0]
b, a




* time-complexity = $O(2^m)$

* using Memoization

→ from the tree in the recursion diagram, we fill the table.

→ Instead of making func^n calls again we use the table values

→ time complexity = $O(m \times n)$ where m & n are sizes of strings.

| | a | b | c | d | \0 |
|---|---|---|---|---|---|
| | 0 | 1 | 2 | 3 | 4 |
| b 0 | 2 | 2 | | | |
| d 1 | 1 | 1 | 1 | 1 | |
| \0 2 | 0 | 0 | 0 | | 0 |

* using dynamic programming - (Tabulation)

→ A: b d (1 2)
B: a b c d (1 2 3 4)

| | | a | b | c | d |
|---|---|---|---|---|---|
| | 0 | 1 | 2 | 3 | 4 |
| b | 1 | | | | |
| d | 2 | | | | |

if (A[i] == B[j])
LCS[i,j] = 1 + LCS[i-1, j-1]

else
LCS[i,j] = max (LCS[i, j-1], LCS[i-1, j])

| | | a | b | c | d |
|---|---|---|---|---|---|
| | 0 | 1 | 2 | 3 | 4 |
| 0 | 0 | 0 | 0 | 0 | 0 |
| b 1 | 0 | 0 | 1 | 1 | 1 |
| d 2 | 0 | 0 | 1 | 1 | 2 |

~~* time complexity~~ =

// included 0 in matrix just to initialise

places where we went diagonal → d & b

∴ LCS = bd , length = 2

* PSEUDO - CODE

* LCS-LENGTH ( X, Y )

```
m = X.length
n = Y.length
let b[1...m, 1...n] & c[0...m, 0...n] be new tables
                                  (for storing symbols)
                                  (for storing length values)
for i = 1 to m
    c[i,0] = 0
for j = 0 to n
    c[0,j] = 0

for i = 1 to m
    for j = 1 to n

        if (x_i == y_j)
            c[i,j] = c[i-1, j-1] + 1
            b[i,j] = "↖"

        else if c[i-1, j] ≥ c[i, j-1]
            b[i,j] = "↑"
            c[i,j] = c[i-1, j]

        else    c[i,j] = c[i, j-1]
                b[i,j] = "←"

return c & b
```

* time - complexity = $O(m \times n)$

* PRINT-LCS ( b, X, i, j )

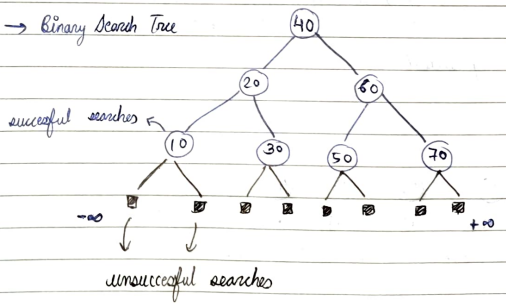
```
if (i == 0) or (j == 0)
    return
if b[i,j] == "↖"
    PRINT-LCS ( b, X, i-1, j-1 )
    print x_i

else if b[i,j] == "↑"
    PRINT-LCS ( b, X, i-1, j )

else
    PRINT-LCS ( b, X, i, j-1 )
```

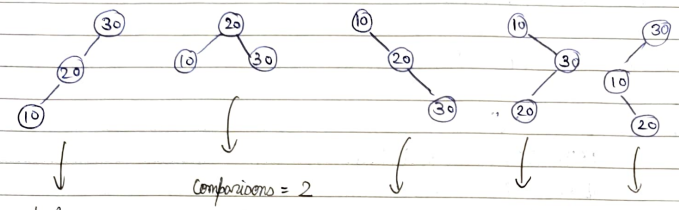
* Optimal Binary Search Tree

→ Binary Search Tree




no. of keys = $n$ , Total BSTs possible = $T(n) = \frac{^{2n}C_n}{n+1}$

Ex- Keys- 10, 20, 30 $T(3) = \frac{6C_3}{4} = \frac{120}{4 \times 6} = 5$




Comparisons = 2 " = 3 " = 3 " = 3

Comparisons for searching = 3

Therefore, this BST is the optimal BST due to less no. of comparisons for searching any key.

* Height of BST ↓ No. of comparisons ↓

---

* if no. of keys = n, no. of unsuccessful nodes = n+1 (external nodes)
* p = successful search probability
  q = unsuccessful " "

Ex- Keys : 10 20 30 40

$p_i$ : $\cdot 1\ p_1$ $\cdot 2\ p_2$ $\cdot 1\ p_3$ $\cdot 2\ p_4$

$q_i$ : $\cdot 1$ $\cdot 05$ $\cdot 15$ $\cdot 05$ $\cdot 05$

$q_0$ $q_1$ $q_2$ $q_3$ $q_4$

$-\infty - 9$ $41 \rightarrow \infty$



| | |
|---|---|
| 1 | 20 |
| 2 | 10, 30 |
| 3 | 40 |

for successful search → level
for unsuccessful search → level - 1

Cost of searching (success + unsuccessful)

$= 1 \times .2 + 2 \times .1 + 2 \times .1 + 3 \times .2$
$+ 2 \times .1 + 2 \times .05 + 2 \times .15 + 3 \times 0.05 + 3 \times 0.05$

$= 2.1$

---

* Formula for cost -

$$Cost[0,4] = \sum_{1 \le i \le n} p_i * level(q_i) + \sum_{0 \le i \le n} q_i * (level(E_i) - 1)$$

$$c[i,j] = \min_{i < k \le j} \{ c[i,k-1] + c[k,j] \} + w[i,j]$$

$$c[0,3] = \min_{0 < k \le 3} \{ c[0,0] + c[1,3] \ , \ c[0,1] + c[2,3] \ , \ c[0,2] + c[3,3] \} + w[0,3]$$

1, 2, 3

$$c[1,3] = \min \{ c[1,1] + c[2,3] \ , \ c[1,2] + c[3,3] \} + w[1,3]$$

$j-i=0$ c[0,0] c[1,1] c[2,2] c[3,3]

$j-i=1$ c[0,1] c[1,2] c[2,3]

$j-i=2$ c[0,2] c[1,3]

$j-i=3$ c[0,3]

$w[0,2] =$ ~~$q_0 q_1$~~ $q_0 + p_1 + q_1 + p_2 + q_2$

$w[0,3] = q_0 + p_1 + q_1 + p_2 + q_2 + p_3 + q_3$

$w[0,3] = w[0,2] + p_3 + q_3$

$$\boxed{w[i,j] = w[i,j-1] + p_j + q_j}$$

Ex- keys = {10, 20, 30, 40}

$p_i$ = {3, 3, 1, 1}

$q_i$ = {2, 3, 1, 1, 1}

$w[0,0] = q_0$
$w[1,1] = q_1$
$w[2,2] = q_2$
$w[3,3] = q_3$
$w[4,4] = q_4$

$r_{ij} \Rightarrow$ root values

↓

i equals to k for which cost is min.

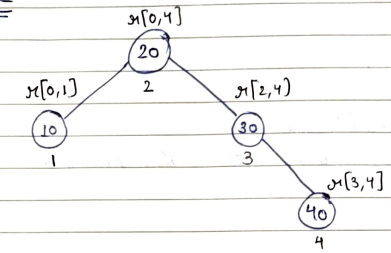
| $j \rightarrow$ | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| $j-i=0$ | $w_{00}=2$ | $w_{11}=3$ | $w_{22}=1$ | $w_{33}=1$ | $w_{44}=1$ |
| | $c_{00}=0$ | $c_{11}=0$ | $c_{22}=0$ | $c_{33}=0$ | $c_{44}=0$ |
| | $r_{00}=0$ | $r_{11}=0$ | $r_{22}=0$ | $r_{33}=0$ | $r_{44}=0$ |
| $j-i=1$ | $w_{01}=8$ | $w_{12}=7$ | $w_{23}=3$ | $w_{34}=3$ | |
| | $c_{01}=8$ | $c_{12}=7$ | $c_{23}=3$ | $c_{34}=3$ | |
| | $r_{01}=1$ | $r_{12}=2$ | $r_{23}=3$ | $r_{34}=4$ | |
| $j-i=2$ | $w_{02}=12$ | $w_{13}=9$ | $w_{24}=5$ | | |
| | $c_{02}=19$ | $c_{13}=12$ | $c_{24}=8$ | | |
| | $r_{02}=1$ | $r_{13}=2$ | $r_{24}=3$ | | |
| $j-i=3$ | $w_{03}=14$ | $w_{14}=11$ | | | |
| | $c_{03}=25$ | $c_{14}=19$ | | | |
| | $r_{03}=2$ | $r_{14}=2$ | | | |
| $j-i=4$ | $w_{04}=16$ | | | | |
| | $c_{04}=22$ | | | | |
| | $r_{04}=2$ | | | | |

DOMS

* cost of tree = 32 i.e. $c_{0,4}$

* generating OBST -

$r[0,4] = 2$




OBST

* Pseudo-Code (e≈c) {time-complexity = $O(n^3)$}

let e[1...n+1, 0...n], w[1...n+1, 0...n] & root[1...n, 1...n] be new tables

```
for i=1 to n+1
    e[i, i-1] = q_{i-1}
    w[i, i-1] = q_{i-1}
for l=1 to n
    for i=1 to n-l+1
        j = i+l-1
        e[i,j] = ∞
        w[i,j] = w[i,j-1] + p_j + q_j
        for r= i to j
            t = e[i,r-1] + e[r+1,j] + w[i,j]
            if (t < e[i,j])
                e[i,j] = t
                root[i,j] = r
return e & root
```

DOMS

* Probabilistic Analysis-

→ uses probability for analysis of algorithm

* The Hiring Problem-

→ ~~suffosa~~ strategy :

→ one candidate each day
→ if new candidate better, then hiring ✓

→ Algo :

```
HIRE(N):
best = 0        // dummy candidate = 0

for i = 01 to N
        interview candidate i
        if candidate i is better than candidate best
                best = i
                hire candidate i
```

* In this analysis, we are bothered of cost over time.

low-cost → interviewing candidate → $C_i$
high-cost → hiring candidate → $C_h$

Let m be the candidates hired from total n.

total cost = $O(nC_i + mC_h)$

* Worst-Case : every next candidate is better than previous

⇒ highest cost

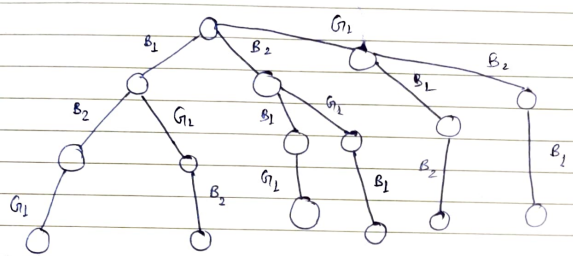
# Backtracking:- uses Brute-Force Approach, uses DFS

Ex- $B_1, B_2, G_1$ — 2 boys, 1 Girl



$n = 3 \Rightarrow 3! = 6$

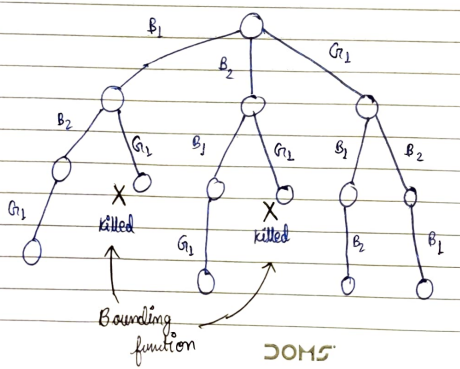
solution can be represented as a tree → state-space tree




∴ no. of leaf nodes = no. of possibilities

& by following any valid path of tree we get answer.

* adding constraint : girl shouldnt sit in middle.




DOMS

* Branch & Bound ⇒ uses BFS, priority queue generates level wise.

→ instead of reaching the leaf node at each level, it finishes generation of level.

---

* Sum of subsets -

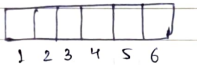
→ given : weights

→ To do : sum of subsets should be any particular value.

∴ $w[1:6] = \{5, 10, 12, 13, 15, 18\}$ ⇒ sum of weights = 73

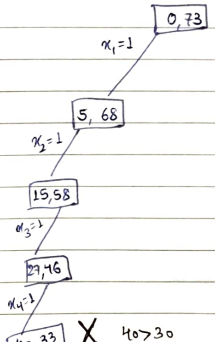
$n = 6$, $m = 30$ → needed sum

$x$




$x_i = 0/1$ ⇒ exclusion / inclusion of weight in soln.

→ By using state space tree




* Bounding functions

$$\sum_{i=1}^{k} w_i x_i + w_{k+1} \leq m$$

$$\sum_{i=1}^{k} w_i x_i + \sum_{i=k+1}^{n} w_i > m$$

DOMS

## * Algorithm-

{ time-complexity : $O(2^n)$ }

* s = sorted set of numbers / weights

```
Backtracking (s, target, start, current_subset, result) :
{
    current_sum ← sum( current_subset )

    if (current_sum == target )
        result.append ( current_subset )

    if ( current_sum > target )
        return ;

    for (i = start to s.length - 1 )
        current_subset . append ( s[i] ) .
//call  Backtracking (s, target, i+1, current_subset, result )'
    current_subset. remove ( )   // remove last element
}


def sum_of_subsets ( s, target )
    sort s ;
    result [ ] ;
    backtracking ( s, target, 0, [ ], result ) ;
    return ;
```



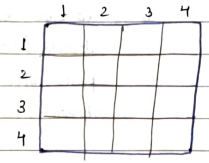
## * N-Queens Problem-

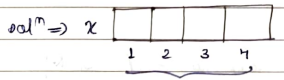
→ give all solutions in which we can place N-Queens on a N×N chess board such that

- no 2 are in same row
- " same column
- " same diagonal

→ give all possible solutions

$Q_1$ $Q_2$ $Q_3$ $Q_4$




## * Algorithm-

```
N_Queens (n)
{   board [n][n] ;
    result [ ] ;
    place_queen ( board, 0, n, result)
    return result }

is_safe ( board, row, col, n)
{   for(i = 0 to i = row - 1 )
        if ( board[i][col] has queen )
            return false

    for( i = row - 1 , j = col - 1  to  i = 0 , j = 0 )

        if ( board[i][j] has queen )
            return false

    for (i = row - 1 , j = col - 1  to  i = 0 , j = n - 1 )
        if ( board[i][j] has queen )
            return false

    return True }
```

```
place_queen ( board, row, n, result )

    if (row == n )
        result.append (board)

    for (col=0 to n-1)
        if ( is_safe (board, row, col, n) )
            board [row][col] ← 'Q'
            place_queen ( board, row+1, n, result )
            board [row][col] ← '\0'
```

* Time-complexity = O(N!)

## * Graph-Coloring Problem-

→ Aim: to color vertices of the graph such that no two adjacent vertices ~~are~~ have same colour.

```
graph_coloring ( graph, m )
    m ← no. of vertices of graph
    colors[m] ← 0                    // Initialise with 0
    if ( assign_colors (graph, m, colors, 0) )
        RETURN colors
    else
        RETURN "No sol" exists"
```

* m = no. of colors



```
assign_colors (graph, m, colors, vertex)
    if vertex == N
        return true                  // all vertices colored

    for color = 1 to m
        if (is_safe (graph, colors, vertex, color))
            colors [vertex] = color
            if (assign_colors( graph, m, colors, vertex+1 ))
                { return true }
            colors [vertex] = 0
```

```
is_safe (graph, colors, vertex, color)
    for all adjacent_vertex to vertex
        if (colors [adjacent_vertex] == color):
            return false
    return true
```

## * Branch & Bound-

* 3 methods:

→ FIFO_BB
→ LIFO_BB
→ LC_BB

↳ Least Cost Branch & Bound
↳ solves the node with least cost first.

Job sequencing

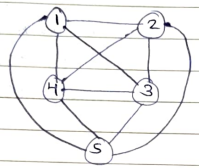
Jobs = { $J_1$, $J_2$, $J_3$, $J_4$ }
P = { 10, 5, 8, 3 }
d = { 1, 2, 1, 2 }

## * Travelling Salesman Problem - (using Branch & Bound)

→ given, a directed graph,

find the least cost path for salesman to travel to all nodes & back.

Ex- reduce the cost matrix :




→ write the min. of each row
→ subtract that min from all elements of row

→ add all the row minima

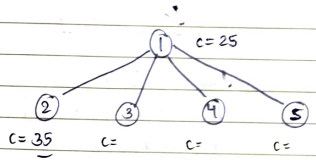
→ similarly, do all these steps for column
→ add the column & row min.

| | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | ∞ | 20 | 30 | 10 | 11 |
| 2 | 15 | ∞ | 16 | 4 | 2 |
| 3 | 3 | 5 | ∞ | 2 | 4 |
| 4 | 19 | 6 | 18 | ∞ | 3 |
| 5 | 16 | 4 | 7 | 16 | ∞ |

cost matrix

* reduced cost = 25

* set upper = ∞




| | | | | |
|---|---|---|---|---|
| ∞ | 10 | 17 | 0 | 1 |
| 12 | ∞ | 11 | 2 | 0 |
| 0 | 3 | ∞ | 0 | 2 |
| 15 | 3 | 12 | ∞ | 0 |
| 11 | 0 | 0 | 12 | ∞ |

reduced - matrix



→ now, consider links of node 1 → 2
→ replace all the elements of row 1 & column 2 with ∞ in the reduced matrix.

* once we come to vertex 2, we shouldnt go back to vertex 1,
∴ matrix [2,1] = ∞

* check if all rows & columns have 0.

∴ cost of node 2 = cost(1,2) + r + r̂

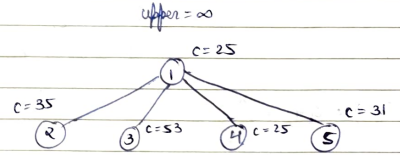
- cost(1,2) : from original matrix
- r : cost of reducing original matrix
- r̂ : any additional cost for reduction.

= 10 + 25 + 0
c = 35

* r̂ is 0, ∵ all rows & columns have 0.

Similarly, we find out for vertex link 1→3 & 1→4 & 1→5

* After- calculation :




→ at this stage, if we use the nodes in the order 2,3,4,5 to further expand, it is called FIFO approach.

* If we use the least cost live node, its called Least Cost Branch & Bound approach (LCBB)

⇒ upon selection of LCBB approach, we explore child nodes of 4.

→ we have chosen node 4.

→ Now, we use the matrix reduced at node 4. * not the initial

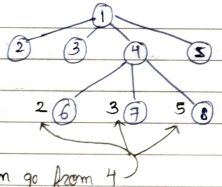
→ graph at 4 ⇒

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ 12 & \infty & 11 & \infty & 0 \\ 0 & 3 & \infty & \infty & 2 \\ \infty & 3 & 12 & \infty & 0 \\ 11 & 0 & 0 & \infty & \infty \end{bmatrix}$$

∴ row 4, column 2 ⟷ ∞

2,4 ← ∞

2,1 ← ∞




* vertices we can go from 4

∴

$$\begin{bmatrix} \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & 11 & \infty & 0 \\ 0 & \infty & \infty & \infty & 2 \\ \infty & \infty & \infty & \infty & \infty \\ 11 & \infty & 0 & \infty & \infty \end{bmatrix}$$

rows & columns have atleast one zero.

∴ cost = c(4,2) + c(4) + $\hat{r}$

= ~~25~~ + 3 + 0 = 28



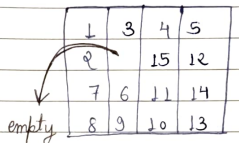
→ similarly, continue the process until reaching leaf node.

→ for the above, leaf cost = 28

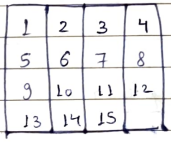
∴ update upper = 28

* Now check every node for cost lesser than upper.
* If $n^{th}$, then gets killed.

* 15 Puzzle Problem - (using branch & bound)




an arrangement




goal arrangement

* The problem states that we need to achieve the goal arrangement from the initial arrangement.

* Constraints:

→ The boxes (empty) can only move up, down, right & left i.e. the empty slot can move (interchange) with the boxes to the left, right, up or down.

* permutation problem to find least cost.

* there are 16! permutations for the above.

* Estimated Cost : $\hat{c}(x) = f(x) + \hat{g}(x)$

* $\hat{c}(x)$ : estimated min cost to reach goal node.
  $f(x)$ : is the length of the path from the root to node 'x'
  $\hat{g}(x)$ : is an estimate of length of shortest path from x to a goal node in the subtree with root x.

* $\hat{g}(x)$ = no. of non-blank tiles not in their goal position.

* At every level, count $\hat{g}(x)$ for each node.

---

* 0/1 Knapsack problem.

given objects along with their ⇒

| | 1 | 2 | 3 | 4 | |
|---|---|---|---|---|---|
| profit | 10 | 10 | 12 | 18 | m = 15 |
| weight | 2 | 4 | 6 | 9 | n = 4 |

To do: fill such that weight ≤ m & profit = max.

* object can either be taken (1) or not (0), nothing in between.

* $u = \sum_{i=1}^{m} p_i x_i \leq m \Rightarrow$ upper bound

  $c = \sum_{i=1}^{m} p_i x_i$ (with portion)
  ↳ cost

---

* Branch & Bound Approach.

knapsack ( p, w, n, m, $I_1$, $I_2$ )

// $p[i]/w[i] \geq p[i+1]/w[i+1]$ , $i \in [1, n)$

$I_1 = I_2 = \phi$
$q = lbb(\phi, \phi)$
k = largest j such that $w[1] + ____ + w[j] < m$ // $\sum_{i=1}^{j} w[i] < m$

for i = 1 to i = k
  if ( $ubb(\phi, \{i\}) < q$ )
    $I_1 = I_1 \cup \{i\}$
  else if ( $ubb(\phi, \{i\}) > q$ )
    $q = lbb(\phi, \{i\})$

* $ubb(I_1, I_2)$ is an upper bound on the optimal value of m with added constraints $x_i = 1$ if $i \in I_1$ & $x_i = 0$

for i = k+1 to i = n
  if ( $ubb(\{i\}, \phi) < q$ )
    $I_2 = I_2 \cup \{i\}$
  else if ( $lbb(\{i\}, \phi) > q$ )
    $q = lbb(\{i\}, \phi)$

* similarly, $lbb(I_1, I_2)$ is lower bound under constraints of $I_1$ & $I_2$.

## * using DP -

```
DPknapsack( n, W, weights[1...n], values[1...n] )
    dp[n+1][W+1]

// dp[i][ω] represents max-value achievable using the first i items
   & a weight limit of ω.

for i=0 to i=n                    // all values of row 0 & col 0 set to 0.
    for W=0 to ω=W
        if i==0 or ω==0
            dp[i][ω] = 0

for i=1 to i=n
    for ω=1 to ω=W
        if weights[i] <= ω
            dp[i][ω] = MAX( dp[i-1][ω], dp[i-1][ω-weights[i]] )
                                                + values[i] )
        else
            dp[i][ω] = dp[i-1][ω]

// Max val ← dp[n][W]

sel = []
ω = W
for i=n down to i=1
    if dp[i][ω] != dp[i-1][ω]
        ADD i to sel
        ω = ω - weights[i]

RETURN sel, dp[n][W]
```

## # Greedy Algorithms -

* optimal substructure : optimal solution of the original problem incorporate optimal sol$^n$ of related subproblems.
* Evaluate / compute all possible ways to parenthesize, then find the optimal way.
* overlapping ~~substructure~~ subproblems : storing and reusing values instead of recomputing.

## * Activity Selection Problem -

→ goal : to schedule several computing activities that require exclusive use of a common resource, to produce a maximum size set of mutually compatible activities.

$S_{ij}$ → set of activities
→ all activities that start after $a_i$ finishes
→ all activities that finishes before $a_j$ starts

$A_{ij} = A_{ik} + A_{kj}$

$A_{ij}$ is optimal sol$^n$ for set $S_{ij} \Rightarrow A_{kj}$ is also optimal

$c[i,j]$ be size of the optimal sol$^n$

$$c[i,j] = \begin{cases} 0 & , S_{ij} = \emptyset \\ c[i,k] + c[k,j] + 1 & , S_{ij} \neq \emptyset \end{cases}$$

given a set of activities (their start time, their finish time), find a max. size subset of mutually compatible activities. (optimal sol$^n$)

$a[i,k] + a[k,j] + 1$

* we have overlapping subproblems. → 

•

* greedy choice is part of optimal solution.

* sorted based on finish time
* there can be many optimal sol$^n$.

* greedy choice might not be in all such sol$^n$.

* Greedy Choice Theorem: consider any non-empty subproblem $S_k$ and let $a_m$ be an activity in $S_k$ with the earliest finish time. then $a_m$ is included in some maximum size subset of mutually compatible activities of $S_k$.

* Proof:

→ let $A_k$ be the maximum size subset of mutually compatible activities in $S_k$.

→ let $a_j$ be the activity in $A_k$ with earliest finish time.

→ If $a_j = a_m$, then $a_m$ is in same max. size subset (MSS) of $S_k$.

→ If $a_j \neq a_m$, then if we substitute $a_m$ for $a_j$, we get another optimal solution.

$A_k' = A_k - \{a_j\} + \{a_m\}$

as $A_j$ is $a_j$ is the first activity in $A_k$ to finish, $a_m$ is the first activity in $S_k$ to finish, $f_j \geq f_m$. $|A_k'| = |A_k|$, $A_k'$ is MSS.

* Correctness Proof:-

// find the first activity in $S_k$ to finish

```
RASP(s, f, k, n)        // sorted; all activities after a_m finishes; n = total no. of activities
m = k+1
while (m ≤ n and s[m] < f[k])
    m = m+1
if (m ≤ n)
    return a_m ∪ RASP(s, f, m, n)
return ∅
```

Recursive Activity Selector

$T(n) = \Theta(n)$ ∴ it is $O(n)$

* { fictitious activity $a_0$, $f_0 = 0$ }

```
GAS(s, f)
n = s.length()
A = {a_1}
k = 1
for m = 2 to n
    if s[m] ≥ f[k]
        A = A ∪ {a_m}
        k = m
return A
```

Greedy Activity Selector

$T(n) = \Theta(n)$ ∴ $O(n)$


DOMS

## * Huffman Coding Algorithm

→ Message : BCCABBDDAECCBBAEDDCC → length = 20

ASCII → 8-bit system

| | | |
|---|---|---|
| A | 65 | 01000001 |
| B | 66 | 01000010 |
| C | 67 | 01000011 |
| D | 68 | 01000100 |
| E | 69 | 01000101 |

8 × 20 = 160 bits

∴

| * character | count / frequency | | code |
|---|---|---|---|
| A | 3 | 3/20 | 000 |
| B | 5 | 5/20 | 001 |
| C | 6 | 6/20 | 010 |
| D | 4 | 4/20 | 011 |
| E | 2 | 2/20 | 100 |

→ Total no. of bits = $\underbrace{20 \times 3}_{\text{message}} + \underbrace{5 \times 8 + 5 \times 3}_{\text{character + code}}$

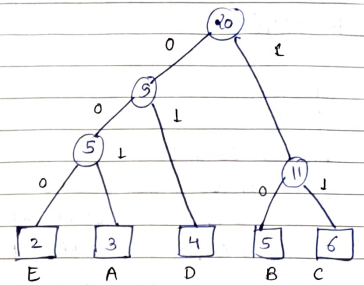
= 60 + 40 + 15

= 115 bits

*

| char | count | code | |
|---|---|---|---|
| A | 3 | 001 | 3 × 3 = 9 |
| B | 5 | 110 | 5 × 2 = 10 |
| C | 6 | 11 | 6 × 2 = 12 |
| D | 4 | 01 | 4 × 2 = 8 |
| E | 2 | 000 | 2 × 3 = 6 |

45 bits




total no. of bits = 45 + $\underbrace{(8 \times 5 + 12)}_{\text{tree/table}}$

= 45 + 52 = 97 bits

* $\sum d_i * f_i \Rightarrow$ total no. of message bits

$d_i$ → distance from root to leaf

$f_i$ → frequency

* Decoding :

→ decode using the tree and if leaf node is encountered start again from the root.

HUFFMAN (C) : { T(n) = O(nlogn) }

n = |C| // C → set of n characters
Q = C // Q → min-priority queue
for i = 1 to n-1
allocate a new node z
z.left = x = EXTRACT-MIN (Q)
z.right = y = EXTRACT-MIN (Q)
z.freq = x.freq + y.freq.

INSERT (Q, z)

return EXTRACT-MIN (Q) // returns the root

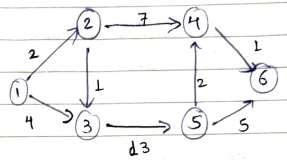
* Difference b/w dp & greedy-

→ overlapping subproblems } dp
→ smallest subproblem

→ greedy choice } greedy
→ largest subproblem

* Shortest Path Problem- (Single Source)

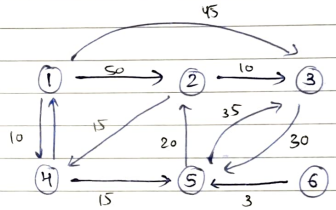
→ shortest path from source to z is the sum of shortest path from source to intermediary x and intermediary to vertex.




* Relaxation

if ( d[u] + c(u,v) < d[v] )
d[v] = d[u] + c(u,v)

* Dijkstra's Algorithm { may or may not work for -ve weights }




starting vertex 1-

| selected vertex | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| 4 | 50 | 45 | (10) | ∞ | ∞ |
| 5 | 50 | 45 | (10) | (25) | ∞ |
| 2 | (45) | 45 | (10) | (25) | ∞ |
| 3 | (45) | (45) | (10) | (25) | ∞ |
| 6 | (45) | (45) | (10) | (25) | ∞ |

* $G = (V, E)$ weighted directed graph
  → non-negative

$w(u,v) \geq 0 \quad \forall$ edges $\in E$

→ The algorithm repeatedly selects the vertex $u \in V - S$ with min. shortest distance estimate, adds $u$ to $S$, and relaxes all edges leaving $u$.

→ using min-priority queue $Q$ of vertices, keyed by their $d$ values.
↓
distance

DIJKSTRA $(G, w, s)$

INITIALISE-SINGLE-SOURCE $(G, s)$
$S = \emptyset$
$Q = G.V$
while $Q \neq \emptyset$
  $u$ = EXTRACT-MIN$(Q)$
  $S = S \cup \{u\}$
  for each vertex $v \in G.Adj[u]$
    RELAX $(u, v, w)$

→ if $(v.d > u.d + w(u,v))$
  $v.d = u.d + w(u,v)$
  $v.\pi = u$

$v.d$ = estimate distance
$v.\pi$ = predecessor

* Time-Complexity = $O((V+E)\log V)$

* Knapsack Problem - {Fractional}

* 0/1 knapsack problem fails in greedy-approach
  → does not fulfill the greedy choice property
  → " " " " optimal substructure property.

∴ Fractional using greedy :

$n = 7$ → objects : 0 1 2 3 4 5 6 7
$m = 15$ profits : P 10 5 15 7 6 18 3
weights : W 2 3 5 7 1 4 1
P/W 5 1·3 3 1 6 4·5 3

↓
max. weight limit

solution $x$ ( 1, 2/3, 1 0 1 1 1 )
$x_1, x_2, x_3, x_4, x_5, x_6, x_7$

* Constraint - $\sum x_i w_i \leq m$
* Objective - $\max \sum x_i p_i$

$\sum x_i w_j = 1 \times 2 + \frac{2}{3} \times 3 + 1 \times 5 + 0 \times 7 + 1 \times 1 + 1 \times 4 + 1 \times 1$
$= 2 + 2 + 5 + 0 + 1 + 4 + 1 = 15$

$\sum x_i p_i = 1 \times 10 + \frac{2}{3} \times 5 + 1 \times 15 + 1 \times 6 + 1 \times 18 + 1 \times 3$
$= 54.6$

Total Profit.

# Divide & Conquer-

→ problem solving technique to solve computational problems.

→ Divide → Conquer → Combine

→ Recursion: Problem solving technique where the solutions of problem depends on solutions of smaller instances of same problem.

- Base-case
- recurrence relation

## * Binary Search-

→ Input : sorted array , target
→ Output : index of target or 'NOT PRESENT'

* Binary-search ( A, start, end, target )

* iterative
* T(m) = O(logm)

```
int mid = (start + end) / 2 ;
while ( start ≤ end )
{   if(A[mid] == target ) { return mid }

    else if ( A[mid] > target )
        { end = mid - 1 }

    else { start = mid + 1 }

    mid = (start + end) / 2
}
```



## * Recursive-

Binary_search (A, start, end, ~~val~~ target) → T(m)

```
c1   if ( start ≤ end )
c2       mid = (start + end) / 2
c3       if (A[mid] == target ) { return mid } c4
c5       else if ( A[mid] > target )
c6 = T(m/2)      { Binary_search ( A, start, ~~end~~ mid -1 , target ) }
c7       else
c8 = T(m/2)      { Binary_search ( A, ~~start~~ mid + 1, end, target) }
```

T(m) = c1 + c2 + c3 + c4 + c5 + c6 + c7 + c8

∴ T(m) = T(m/2) + C → Recurrence relation

T(m/2) = T(m/2²) + C

T(m/2²) = T(m/2³) + C

solving using back substitution.

T(m/2^(k-1)) = T(m/2^k) + C → Base-case reached

Add

T(m) = T(m/2^k) + kC

m/2^k = 1

T(m) = T(1) + kC

* k = log₂ m

T(m) = C + kC

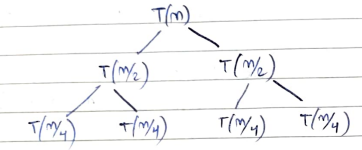
T(m) = ~~(k+1)C~~ (k+1)C

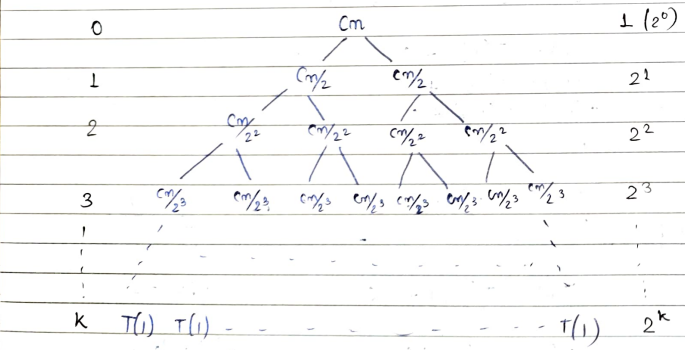
T(m) = (1 + logm) c

* $T(n) = O(\log n)$

* ~~Method~~ Recursion tree Method

* $T(n) = 2T(n/2) + c$




| level | Nodes | Cost |
|---|---|---|
| 0 | $2^0$ | $cn$ |
| 1 | $2^1$ | $2(cn/2) = cn$ |
| 2 | $2^2$ | $2^2(cn/2^2) = cn$ |
| ⋮ | ⋮ | ⋮ |
| k | $2^k$ | $2^k(cn/2^k) = cn$ |

Total cost = $(k+1)\,cn$

$\because$ $k = \log n$

$T(n) = (k+1)cn = (\log n + 1)cn = cn\log n + cn$

* $\boxed{T(n) = O(n \log n)}$

* Merge-Sort :

Merge_Sort ( A, p, q ) — p: start, q: end

```
if (p<q)                          } Θ(1)
    mid = (p+q)/2
    Merge_sort (A, p, mid)        → T(n/2)
    Merge_sort (A, mid+1, q)      → T(n/2)
    Merge (A, p, mid, q)          → Θ(n)
```

* extra space required → $\Theta(n)$

* $T(n) = 2T(n/2) + \Theta(n)$ , $n > 1$

  $T(n) = c$ , $n = 1$

Merge (A, p, mid, q)

$n_1 = mid - p + 1$

$n_2 = q - mid$

let $L[1 \ldots n_1+1]$ & $R[1 \ldots n_2+1]$ be ~~new~~ subarrays

```
for i=1 to n1                          } θ(n)
|   L[i] = A[p+i-1]
for j=1 to n2
|   R[j] = A[mid+j+1]

L[n1+1] = ∞                            } c
R[n2+1] = ∞
i = 1                                  } c
j = 1

for k = p to q                         } θ(n)
    if (L[i] ≤ R[j])
    |       A[k] = L[i]
    |       i = i+1
    else
    |       A[k] = R[j]
    |       j = j+1
```



## * Multiplication of two integers -

```
Multiply (a[1...p], b[1...q], base)
    for i=1 to q                                        C1   m+1
    |   carry = 0                                      C2   m
    |   for j=1 to p                                    C3   Σ_1^m m
    |   |   prod[i+j-1] += carry + a[j]·b[i]           C4   "
    |   |   carry = prod[i+j-1] / base                 C5   "
    |   |   prod[i+j-1] %= base                        C6   "
    |   prod[i+p] += carry                              C7   m
    return p                                            C8   1
```

$\therefore T(m) = C_1(m+1) + C_2 m + (C_3 + C_4 + C_5 + C_6) m^2 + C_7 m + C_8$

$T(m) = Am^2 + Bm + C$

$\boxed{T(m) = \Theta(m^2)}$ → ~~$\{p \approx q \approx m\}$~~ $\{p = q = m\}$

## * Divide & Conquer Approach -

ex- $a = 2345$, $n$ = no. of digits = 4

$b = 3216$

→ divide a & b into 2 parts.

$a_L$ $a_R$

2 3 4 5

$b_L$ $b_R$

X 3 2 1 6

→ Conquer : recursively solve subproblems

$a = 2345 = 23 \times 10^2 + 45$

$b = 3216 = 32 \times 10^2 + 16$

$(a \cdot b) = (23 \times 10^2 + 45)(32 \times 10^2 + 16)$

$= 23 \cdot 32 \cdot 10^4 + 23 \cdot 16 \cdot 10^2 + 45 \cdot 32 \cdot 10^2 + 45 \cdot 16$

$$a \cdot b = \underbrace{a_L \cdot b_L \cdot (base)^n}_{①} + \underbrace{(a_L \cdot b_R + b_L \cdot a_R)(base)^{n/2}}_{②,③} + \underbrace{a_R \cdot b_R}_{④}$$

$P_1 = a_L b_L$ $\quad P_3 = a_R b_L$

$P_2 = a_L b_R$ $\quad P_4 = a_R b_R$

$$P = P_1 \times base^n + (P_2 + P_3) base^{n/2} + P_4$$

Multiply (A, B) $\quad T(n)$

$n = max(size\ of\ A, size\ of\ B)$ } c

if $n == 1$ {return $a \cdot b$ }

$P_1 = Multiply(a_L, b_L)$

$P_2 = Multiply(a_L, b_R)$

$P_3 = Multiply(a_R, b_L)$ } $T(n/2)$

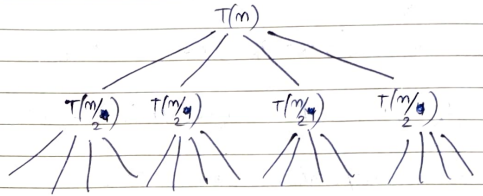
$P_4 = Multiply(a_R, b_R)$

DOMS

$P = P_1 \cdot base^n + (P_2 + P_3) \cdot base^{n/2} + P_4$ } $\Theta(n)$

return P → $\Theta(1)$

$T(n) = 4T(n/2) + \Theta(n)$




| level | Nodes | Cost |
|---|---|---|
| 0 | $4^0$ | $cn$ |
| 1 | $4^1$ | $c(n/2) \cdot 4$ |
| 2 | $4^2$ | $c(n/2^2) \cdot 4^2$ |
| ⋮ | ⋮ | ⋮ |
| k | $4^k$ | $c(n/2^k) \cdot 4^k = cn \cdot 2^k$ |
| | | $= cn \cdot n$ |
| | | $= cn^2$ |

$k = \log n$

Total Cost = $T(n) = cn + 2cn + 2^2cn + 2^3cn + \dots = cn^2$

$$T(n) = O(n^2)$$

DOMS

* if cost at each level is same ⇒ T(n) = height · cost
* if cost is ↑ ⇒ no. of nodes at last level · cost
* " " ↓ ⇒ cost of the root

---

* In previous relation ↓

$$a \cdot b = a_L \cdot b_L \cdot base^n + (a_L \cdot b_R + a_R b_L)\, base^{n/2} + a_R b_R$$

by replacing $(a_L b_R + a_R b_L)$ with $(a_L + a_R)(b_L + b_R) - a_L b_L - a_R b_R$

we reduce the no of multiplications from 4 to 3.

---

* KARATSUBA (A, B)

```
n = max-(size of A, size of B)
if n == 1                                   } c
    return A[1].B[1]

P1 = KARATSUBA (AL, BL)
P2 = KARATSUBA (AL + AR, BL + BR)           } 3 · T(n/2)
P3 = KARATSUBA (AR, BR)

P = P1 · base^n + (P2 - P1 - P3) base^(n/2) + P3      } θ(n)
return P
```

$\therefore T(n) = 3 \cdot T(n/2) + \theta(n)$

$$T(n) = c \cdot 3^k = c \cdot 3^{\log_2 n} = c \cdot n^{1.59} = \theta(n^{1.59})$$

$$\boxed{T(n) = O(n^{\log_2 3})}$$

DOMS

## * Square Matrix Multiplication

* Input : 2 square matrices A & B
* Output : Product matrix of A & B

```
SMM(A, B)
    n = size of A
    for i = 1 to n
        for j = 1 to n
            ~~for k = 1 to n~~
                c_ij = 0
            for k = 1 to n
                c_ij += A_ik . B_kj
    return C
```

* $T(n) = O(n^3)$

---

## * Divide & Conquer Approach -

Ex -

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 3 & 4 & 1 & 2 \\ 1 & 2 & 4 & 3 \\ 3 & 4 & 1 & 1 \end{bmatrix} \quad \begin{bmatrix} 5 & 6 & 7 & 8 \\ 9 & 10 & 2 & 3 \\ 1 & 3 & 2 & 5 \\ 5 & 6 & 0 & 9 \end{bmatrix}$$

4×4 4×4

$A_{11}$ $A_{12}$ $A_{21}$ $A_{22}$ — A

$B_{11}$ $B_{12}$ $B_{21}$ $B_{22}$ — B

DOMS

SMM(A, B)

n = A.rows
let C be a new n×n matrix
if n == 1
    $c_{11} = a_{11} \cdot b_{11}$

} $\Theta(1)$

else

$c_{11} = SMM(A_{11}, B_{11}) + SMM(A_{12}, B_{21})$
$c_{12} = SMM(A_{11}, B_{12}) + SMM(A_{12}, B_{22})$
$c_{21} = SMM(A_{21}, B_{11}) + SMM(A_{22}, B_{21})$
$c_{22} = SMM(A_{21}, B_{12}) + SMM(A_{22}, B_{22})$

(matrix addition)

} $8T(n/2) + \Theta(n^2)$

return C

* $T(n) = 8T(\frac{n}{2}) + \Theta(n^2)$

$\Rightarrow$ $T(n) = O(n^3)$

* STRASSEN's Matrix Multiplication

$P = (A_{11} + A_{22})(B_{11} + B_{22})$
$Q = (A_{21} + A_{22})(B_{11})$
$R = A_{11}(B_{12} - B_{22})$
$S = A_{22}(B_{21} - B_{11})$
$T = (A_{11} + A_{12})B_{22}$
$U = (A_{21} - A_{11})(B_{11} + B_{12})$
$V = (A_{12} - A_{22})(B_{21} + B_{22})$

$C_{11} = P + S - T + V$
$C_{12} = R + T$
$C_{21} = Q + S$
$C_{22} = P + R - Q + U$



$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$
$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$
$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$
$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$

* $T(n) = \begin{cases} 1 & , n \leq 2 \\ 7 \cdot T(n/2) + n^2 & , n > 2 \end{cases}$

* $\boxed{T(n) = O(n^{\log_2 7}) = O(n^{2.81})}$

* Maximum Subarray Sum (Contiguous subarray)

Max_subarray (A)

* $T(n) = O(n^3)$

```
max_sum = 0
begin_index = 0
end_index = 0
for i = 1 to A.length()                    // i represents size of selected subarray
    for j = 1 to (A.length() - i + 1)      // j represents index
        if (j + i - 1) > A.length()
            break

        sum = 0
        for k = j to i+j-1                 // k for sum
            sum += A[k]
        if (sum > max_sum)
            max_sum = sum
            begin_index = j
            end_index = i+j-1
```

* Better Approach -

```
max_subarray (A)

    max_sum = 0
    begin_index = 0
    end_index = 0

    for index = 1 to A.length()
        sum = 0
        for size = 1 to A.length()
            if (index + size - 1) > A.length()
                break
            sum += A[index + size - 1]
            if sum > max_sum
                max_sum = sum
                begin_index = index
                end_index = index + size - 1
```

* $T(n) = O(n^2)$

---

* using Divide & Conquer -

```
MSS(A, l, h)
    if l == h
        return l, h, A[l]
    else
        mid = ⌊(l+h)/2⌋
```



(ll, lh, ls) = MSS(A, l, mid) ⟶ Left
(rl, rh, rs) = MSS(A, mid+1, h) ⟶ Right
(cl, ch, cs) = C_MSS(A, l, mid, h) ⟶ Crossing

(ll, lh, ls: low, high, sum)

```
if ls > rs & cs
    return (ll, lh, ls)
else if rs > ls & cs
    return (rl, rh, rs)
else
    return (cl, ch, cs)
```

```
C_MSS(A, l, mid, h)
    ls = -∞ , sum = 0
    for i = mid down to l :
        sum += A[i]
        if sum > ls
            ls = sum
            ll = i
    rs = -∞ , sum = 0
    for j = mid+1 to h
        sum += A[j]
        if sum > rs
            rs = sum
            rh = j
    return (ll, rh, ls+rs)
```

* Crossing - Maximum Subarray Sum

* $T(n) = O(n \log n)$

## * Kadane's Algorithm

* $T(n) = O(n)$

```
MSS(A)

    res = A[0]
    maxending = A[0]

    for i = 1 to (A.length() - 1)
        maxending = max(maxending + A[i], A[i])
        res = max(res, maxending)

    return res
```

* find the max. sum ending at index i by either
  - → extending the max. sum subarray ending at index i-1 or by
  - → starting a new subarray from index i
* update res if max. subarray sum ending at index i > res



## * Master Theorem -

### * Basic - Master Method

→ $T(n) = a\,T(n/b) + f(n)$ , $a \geq 1$ , $b > 1$

→ case-1 : if $f(n) = O(n^{\log_b a - c}) \Rightarrow T(n) = \Theta(n^{\log_b a})$
{ $n^{\log_b a} > f(n)$ }

→ case-2 : if $f(n) = \Theta(n^{\log_b a}) \Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$
{ $n^{\log_b a} > f(n)$ }

→ case-3 : if $f(n) = \Omega(n^{\log_b a + c}) \Rightarrow$ check condⁿ : $a \cdot f(n/b) \leq c \cdot f(n)$

$T(n) = \Theta(f(n))$

### * standard master method

* $T(n) = a\,T\left(\frac{n}{b}\right) + n^k \cdot \log_m p$ (+ve)

* solution : $T(n) = n^{\log_b a} \times [U(n)]$
  - → $U(n)$ depends on $h(n)$ → if $h(n)$ $U(n)$

$h(n) = \frac{f(n)}{n^{\log_b a}}$

$n^n$, $n > 0$ → $O(n^n)$
$n^n$, $n < 0$ → $O(1)$
$(\log n)^i$, $i \geq 0$ → $\frac{(\log_2 n)^{i+1}}{i+1}$

~~→ if h(n)~~